DNSSEC_Signature_3_working

You are an DNS security expert. Here is an example DNS attack that effects the cache of the resolver to reduce its impact.
Example:
NS cache flush attack
Summary:
Attack owns an auhtoritative server and control example.com. Domain.
The attacker issues A type queries for different unique names under this domain. For example, test1.example.com, test2.example.com test10000.example.com
The attacker's authoritative server responds this queries with around 1500 NS records thus making the DNS reply 65k(the maximum value). Each of these values are cached by the resolver, thus evicting the benign cached records from the resolver's cache causing resolver to requery these domains thus reducing its performance significantly.

I want you to come with another cache flushing attack based on DNSSEC.
utilize your DNSSEC knowledge and check anything can be used against to a resolver for cache flushing purposes.
The aim is to generate 65k DNSSEC responses and have the resolver cache all the data in the answer.

BIND put a max-record limit for any RRSet type as 100. So we need to be more creative to have large responses with 100 records max.

The aim is having 65k answers (1 response) not multiple. Focus on any unbounded field that can serve our purpose here.

Below is a conceptual DNSSEC-based "cache flushing" attack that aims to produce a single 65 KB response (the maximum DNS message size) and force resolvers—especially those with limited cache size—to store an excessive amount of DNSSEC data. This pushes out (evicts) legitimate records from the cache and degrades resolver performance.

High-Level Idea

1. **Own an Authoritative Zone**
   The attacker controls a domain (e.g., `example.com`) and its authoritative name servers.
2. **Craft a 65 KB DNSSEC Response**
   When queried for a particular RR (e.g., `A` or `DNSKEY`) at some subdomain (`big.example.com`), the authoritative server returns:
   - **A minimal 'Question' section** (just to trigger the DNSSEC answer),
   - **A massive 'Answer' (and possibly 'Authority'/'Additional') section** containing DNSSEC-related records (DNSKEY, RRSIG, DS, etc.) whose combined size is ~65 KB.
3. **Exploit Large / "Unbounded" Fields**
   Because BIND (and many other resolvers) imposes a *max-records-per-RRSet limit* (often 100), simply cramming more than 100 records in the same RRSet won't work. Instead, the trick is to maximize the *size* of each record rather than just the *number* of records.
4. **Force the Resolver to Cache It**
   A validating resolver must fetch and cache the DNSSEC data to validate the authenticity of the records. Storing 65 KB of data for a single query can evict legitimate cached entries and degrade resolution performance.

Key DNSSEC Fields to Leverage

Because we are capped at (for instance) 100 records per RRSet in some implementations (e.g., BIND's `max-record-ttl` or similar parameters), we need each record's **RDATA** to be as large as possible. Below are some DNSSEC fields that can be bloated:

1. **DNSKEY Records**
   - **Large RSA keys**: DNSKEY RDATA can hold very large public keys if the zone operator chooses; 4096- or 8192-bit keys are possible. Each DNSKEY RR can be hundreds to over a thousand bytes.
   - **Multiple DNSKEYs** (up to 100). Even if each DNSKEY is just under 1 KB, 100 of them can approach 100 KB—but remember we are hitting overall DNS message size limits

of 64–65 KB (EDNS0).

- **Custom Algorithm**: Technically, you could define a custom (private use) DNSSEC algorithm that allows arbitrarily large keys. Not all resolvers will validate them, but from an "attack" perspective, it may still cause them to parse and cache the record set if they do partial validation or store unrecognized algorithms.

2. **RRSIG Records**
   - For each DNSKEY in the zone, you can generate a signature (RRSIG) over the zone data.
   - **Large RSA signatures**: If you use 4096- or 8192-bit RSA keys, each RRSIG can also be hundreds to over a thousand bytes long.
   - If you have multiple DNSKEYs, each key can produce a separate RRSIG. That is how you can add up to 100 RRSIGs (provided your zone is signed by multiple keys).
3. **NSEC / NSEC3 Records**
   - **Long Salt in NSEC3**: The salt field in NSEC3 can be up to 255 bytes, which is not huge, but might still help.
   - **Multiple NSEC(3) records** for different names. However, hitting the 100-record limit can be an obstacle.
   - **Large Type Bitmap** in NSEC(3): Theoretically you can set many windows with bits set. But in practice, the maximum size of the Type Bitmap is still relatively small compared to DNSKEY or RRSIG bloat.
4. **DS Records**
   - Not as large as DNSKEY or RRSIG, but you can have multiple DS records referencing multiple large DNSKEYs at the child. Each DS has a digest, which can be up to 512 bits (e.g., SHA-512). You could have multiple digest types. However, each DS is still relatively small compared to huge keys or signatures.

Putting It All Together

The crux of the strategy is **fewer RRsets but each record is enormous**. For example:

1. **Create 1–5 DNSKEY RRsets** under `example.com` with each RRset containing up to 100 DNSKEYs.

- Each DNSKEY = RSA 4096 bits or 8192 bits.
- That alone could be tens of KB.

2. **Sign them** with multiple large keys, generating multiple (up to 100) RRSIGs for that RRset.
   - Each RRSIG can be 400–1000+ bytes if using large RSA.
   - Potentially pushing the total to near the 65 KB EDNS0 limit once you add normal DNS overhead.
3. **Include DS records** pointing back and forth to these DNSKEYs (in the Authority section). Possibly each DS record is repeated for multiple hash algorithms.
4. **One Query Trigger**
   - An attacker can have a query for `big.example.com DNSKEY` (or even `big.example.com A` with `DO` bit set, requesting DNSSEC).
   - The authoritative server sends back a single monstrous 65 KB response that includes:
     - The question name and type,
     - The `DNSKEY` RRSet (or `A` + the associated RRSIG, DNSKEY, DS records in additional sections),
     - Enough large RRSIGs to bloat the total size,
     - Possibly some NSEC(3) records with large salt or large type bitmaps.

Because you only need **one** 65 KB answer, you do not necessarily have to saturate the number of RRs to 100 in every single set—just enough to maximize *total* byte size.

Practical Example (Simplified)

1. **Zone:** `example.com`
2. **Key:** `Kexample.com.+008+...` (algorithm 8 = RSA/SHA-256 with a large 4096/8192-bit key).
3. **Have ~50–80 DNSKEYs** each sized ~500–1000 bytes. (Stay below 100 to avoid the BIND cap).

4. **Sign all zone data** with *each* of those keys, producing the same number of RRSIGs.
5. **Potentially add DS records** in the Authority section.

In the end, the single DNS response can approach the EDNS0 65535-byte limit.

> **Note: The attacker must ensure the packet does not exceed the maximum DNS message size (65,535 bytes). While TCP fallback will occur for UDP if it's truncated, the idea is to still keep the single DNS response *just* under 64–65 KB so it all gets cached at once.**

Impact on the Resolver

- When the resolver receives this single huge response, it must:
  1. **Validate** (or attempt to) the signatures, which is CPU-expensive.
  2. **Cache** all RRs from the answer (and authority/additional sections).
  3. Potentially flush out legitimate cached data if its cache memory is limited.
- This reduces performance for normal queries and, if repeated (by issuing queries for many subdomains each returning a 65 KB DNSSEC response), can starve the cache of more important records.

Why This Works Even with BIND's 100-Record Limit

- BIND's `max-records` (often set to 100 by default) **limits the number of records per RRSet**, *not* the total packet size.
- By **maximizing each individual record's size** (large RSA keys, large RSA signatures) rather than just stacking thousands of records, you can still fill the DNS response up to 65 KB with well under 100 RRs in each RRSet.
- Each DNSKEY or RRSIG can be hundreds or thousands of bytes in RDATA. Multiply that by dozens of DNSKEYs/RRSIGs, and you have your 65 KB response.

Summary

Attack Vector

- Control an authoritative zone.
- Publish abnormally large DNSKEYs (and possibly large RRSIGs for each key).
- On a single query (with DNSSEC requested), return a **single, valid** ~65 KB DNS message that the resolver must parse and cache.

Result

- The resolver's cache is flooded with large DNSSEC RRs, evicting benign records.
- Repeated queries worsen the cache churn and performance overhead.

This highlights how DNSSEC's flexibility (especially in key size and multiple parallel signatures) can be abused to generate massive responses, which can be weaponized for cache-flushing attacks—despite BIND's (and other resolvers') record-per-RRSet caps.

Are there other ways ? consider DNS features as well not just DNSSEC

Below are **additional** ideas—beyond DNSSEC key/signature bloat—that can produce *single* large DNS responses (up to the ~65 KB EDNS0 limit) and thereby flush the cache. The core concept is always the same: **force the resolver to store a big blob of data** in one shot. While some resolvers (and authoritative servers) have specific mitigations (e.g., limiting the number of records in an RRSet, blocking large ANY queries, etc.), attackers can still combine these DNS features in clever ways.

---

1. Large TXT (or SPF) Records

- **TXT is basically unbounded** within the 65535-byte maximum RDATA limit.
- A single TXT record can contain multiple 255-byte "chunks," concatenated. Hence, you could have up to ~64 KB of text in *one* record.
- The attacker's zone can publish a TXT record like:

```arduino
big.example.com. IN TXT "Part1_of_255_bytes" "Part2_of_255_bytes" ...
"PartN_of_255_bytes"
```

- Then sign it with **DNSSEC** (optional), producing more RRSIG records and pushing the total DNS response size close to 65 KB.
- Even if the resolver or BIND enforces a **100-record** cap per RRset, that doesn't matter if you use **one** record with extremely large RDATA.

Why it's effective:

- TXT or SPF data is not heavily restricted in size by RFC.
- Resolvers typically *do* cache the TXT record if it's in the Answer section.
- A single query for `TXT big.example.com` (with EDNS0 large buffer) can yield a near 65 KB response that goes into the resolver's cache, evicting benign records.

---

2. Over-Sized SVCB/HTTPS Records

Newer DNS record types like **SVCB** (Service Binding) or **HTTPS** (an alias of SVCB) can contain a list of "key=value" parameters in their RDATA. In principle, each parameter can be large (up to 65535 bytes, though practical limits may vary by implementation).

- Publish an SVCB record:

```arduino
_service._proto.example.com.  IN  SVCB 1 target.example.com. (
    key1="a long string"
    key2="another long string"
    ...
)
```

- You can have up to 100 of these SVCB RRs (if not blocked by the server/resolver). Each RDATA can be quite large, potentially pushing the entire response up to 65 KB when combined with signatures, additional data, etc.

Why it's effective:

- SVCB/HTTPS records are somewhat new, so mitigations might be less mature.
- The “key=value” format allows stashing large amounts of data in each record’s RDATA.
- If the resolver caches them, it can cause significant cache usage.

---

3. Single Record with Very Long Domain Names

Some DNS record types store **domain names** in their RDATA (e.g., MX, SRV, NS, CNAME). You can exploit the fact that each label can be up to 63 bytes, and total domain length can be up to 253 bytes.

- For instance, an `MX` record has the form:

```csharp
big.example.com.  IN MX  10  this-is-a-very-very-very-very...-long-name.example.org.
```

- You can create up to 100 such MX records (before hitting the BIND cap) and use the **maximum domain name length** in each. That yields a large total RDATA size across all 100 MX records, plus overhead.
- Combine with DNSSEC signatures to further inflate the response.

Why it’s effective:

- Each MX (or SRV, or NS) record can hold a full 253-byte domain name in RDATA.
- 100 of these with a 253-byte name each can produce ~25 KB just in these domain references, plus overhead, plus RRSIG bloat.
- Not as huge as a single 64 KB TXT, but still a multi-kilobyte chunk.

---

4. ANY Query (If Not Rate-Limited/Blocked)

Historically, querying `qtype=ANY` for a name with multiple RR sets (TXT, A, AAAA, MX, NS, etc.) could produce a *massive* response. However:

- Modern authoritative servers or resolvers often implement **minimal responses** or **block ANY** queries to mitigate DDoS amplification.
- Some resolvers, if forced to fetch the entire set from the authoritative server, might cache it all.
- Attackers with their own authoritative server could respond with everything (TXT, AAAA, SVCB, DS, DNSKEY, you name it) in the Answer section—pushing the response to 65 KB.

Why it's becoming less effective:

- ANY queries are frequently restricted or responded to with minimal data. Many DNS providers refuse to send huge ANY answers.

---

5. DNAME or CNAME Chaining

- **DNAME** can alias an entire subtree, and CNAME can alias a single name.
- A creative attacker might use nested or chained CNAMEs or a single DNAME referencing a very long target domain.
- Each redirection can add domain names to the Additional/Authority sections, especially if signed with DNSSEC.
- However, there are practical recursion limits (e.g., 16 CNAMEs is a typical cap), so *extreme* chaining is often mitigated.

Why it's limited:

- Each step in the chain might be followed by the resolver, but modern resolvers have **loop** and **depth** detection.
- BIND and others typically limit how many redirections they'll process.

---

6. Large "Glue" in Additional Section

- When returning answers with child delegation, the authoritative server can stuff the Additional section with "glue" addresses (A/AAAA).

- You can publish up to 100 hostnames (the BIND RR count limit) each with multiple AAAAs.
- Although each AAAA is only 16 bytes, if you put enough of them plus large domain names, it starts adding up.
- Combine with DNSSEC signatures, again, to push the total over the top.

Why it's partial:

- By itself, large glue might not reach 65 KB easily. Typically, it's more effective to rely on big RDATA (e.g., TXT) or large DNSSEC keys.
- Some resolvers do **not** cache additional-section glue. They only cache in-bailiwick NS or A/AAAA from the Answer/Authority if needed.

---

7. Multiple or Large SRV Records

- **SRV** RDATA = Priority, Weight, Port, and a Target domain name.
- Like MX, that target domain name can be up to 253 bytes.
- You can have up to 100 SRV records in one RRSet, each pointing to a maximum-length domain name.
- Then sign them, etc.

Why it's similar to MX approach:

- Another way to store large domain names in the RDATA.
- 100 records × 253 bytes each = ~25 KB of domain data, plus overhead.

---

8. Combining Non-DNSSEC Bloat With DNSSEC Bloat

A potent attacker tactic is to **layer** a big non-DNSSEC record set (e.g., a giant TXT or 100 long-domain MX records) **plus** DNSSEC signatures:

1. **Large primary RRSet** (e.g., 50–100 TXT/MX/SRV records)

2. **Multiple DNSKEYs** (say, 10–50 large RSA keys)
3. **Multiple RRSIGs** (one per key → 10–50 signatures)
4. Possibly **NSEC(3)** with large salt/bitmaps

This synergy can produce a single near-65 KB response. Even if you cannot exceed 100 total records in a single RRSet, each record can be *massive*, and the presence of multiple RRSIG records for each set quickly inflates the packet size.

Putting It All Together

Attack Flow

1. **Own Authoritative Zone**
   - e.g., `example.com` with your own authoritative DNS server.
2. **Publish Large RRs**
   - Could be a single giant TXT or SVCB record (~64 KB).
   - Or up to 100 MX/SRV records each referencing 253-byte domain names.
   - Optionally sign them with multiple large DNSKEYs to inflate RRSIG.
3. **Send a Single Query**
   - `dig big.example.com TXT +dnssec` (or MX, SRV, ANY, etc.)
   - The authoritative server returns a **single** ~65 KB response.
4. **Resolver Caches Everything**
   - The resolver must store the entire answer set (and associated DNSSEC) in cache.
   - Cache-limit or memory pressure leads to eviction of legitimate records.
   - Repeated queries or repeated unique subdomains (test1, test2, ...) can push the resolver cache to thrash.

Why It Works

- **Unbounded RDATA** in TXT, SVCB, or large domain references in MX/SRV let you hit the 65 KB max with fewer than 100 RRs (thus bypassing BIND's per-RRSet record limit).

- **DNSSEC** or repeated domain references in RRSIG, DS, or Authority sections add further overhead to balloon the message size.
- The resolver, once it receives that single large response, typically has no choice but to store it all if it's considered valid data.

Final Thoughts

Beyond using **DNSSEC** alone, attackers can leverage various DNS features to assemble *massive* single-response packets. The general pattern is:

1. **Control authoritative data** (own the zone).
2. **Abuse unbounded or large-allowed fields** (TXT, SVCB, domain-name fields in MX/SRV, etc.).
3. **Optionally stack on DNSSEC** to blow up the response size even more.

Such "cache-flushing" or "cache-pollution" attacks remain a challenge, especially for resolvers with smaller cache sizes or without strict per-zone or per-name memory constraints.

> Focus on this. Explain how it can work with the BIND 100 record cap
>
> 4. ANY Query (If Not Rate-Limited/Blocked)
>
> Historically, querying qtype=ANY for a name with multiple RR sets (TXT, A, AAAA, MX, NS, etc.) could produce a massive response. However:
>
> Modern authoritative servers or resolvers often implement minimal responses or block ANY queries to mitigate DDoS amplification.
> Some resolvers, if forced to fetch the entire set from the authoritative server, might cache it all.
> Attackers with their own authoritative server could respond with everything (TXT, AAAA, SVCB, DS, DNSKEY, you name it) in the Answer section—pushing the response to 65 KB.

Why it’s becoming less effective:

ANY queries are frequently restricted or responded to with minimal data. Many DNS providers refuse to send huge ANY answers.

Below is a focused explanation of how a **qtype=ANY** query can still produce a *single large response*—potentially up to 65 KB—even with **BIND’s 100-records-per-RRset cap** in place.

---

1. BIND’s 100-Record Cap Is Per RRset, Not Per Response

- **What the Cap Means**: BIND (and other resolvers) often have a limit on the number of records in a *single RRset* (for example, 100). This prevents a single RRset (e.g., an `A` RRset with thousands of IPs) from ballooning.
- **But ANY Queries Return *All* RRsets**: An ANY query asks for *all* record types associated with the domain— `A`, `AAAA`, `NS`, `MX`, `TXT`, `DNSKEY`, etc.
- **Multiple RRsets**: If each type’s RRset has up to 100 records, you can have *many* RRsets in the same *one* response.
  - e.g., 100 `A` records + 100 `AAAA` records + 100 `TXT` records + 100 `MX` records + 10 `DNSKEY` records + 10 RRSIGs (etc.)
  - Each distinct RRset stays under the 100-record limit—but combined, they can contain hundreds of total records in a single DNS packet.

Hence, the BIND 100-record cap does *not* stop you from returning 100 records for one RRset **plus** another 100 for a second RRset, **plus** another 100 for a third, and so on. As long as each RRset remains below 100, BIND will allow all of them in the same (ANY) response.

---

2. How the Single Response Grows So Large

- **Summation of Multiple RRsets**: ANY queries gather *all* RR types published for that name. If the attacker’s authoritative server publishes numerous distinct RRsets, each just

under 100 records, the final response includes *all* of them.

- **DNSSEC Overhead**: If the zone is signed, each RRset might also have RRSIGs, plus possibly DS or DNSKEY records in the Additional/Authority sections. This *extra overhead* (signatures, keys) can push the total packet size closer to 65 KB.
- **Unbounded RDATA**: Some record types can have large RDATA (e.g., huge `TXT` strings, or large domain names in `MX` / `SRV`). Even if each RRset is only 100 records, each record can be large, further inflating the total packet size.

As a result, it's entirely feasible for a single ANY response to approach the **65,535-byte DNS message limit** even though *no single* RRset has more than 100 records.

---

3. Practical Example

Suppose the attacker controls `big.example.com` and sets up:

- **A** RRset with 100 addresses (each well-formed).
- **AAAA** RRset with 100 IPv6 addresses.
- **TXT** RRset with 2–3 *extremely large* TXT records (still <100).
- **MX** RRset with 20–30 records referencing very long domain names.
- **DNSKEY** RRset with a few large RSA keys.
- **RRSIG** for each RRset.

When an ANY query comes in, the authoritative server returns *all* these sets in **one** response:

1. 100 A records (each small, but total ~100 records).
2. 100 AAAA records (another ~100 records).
3. A handful of large TXT records (still under 100, but big RDATA).
4. 20–30 MX records (long domain names).
5. DNSKEY + multiple RRSIGs (possibly large if using big RSA keys).
6. Potentially DS records or other types.

Even though each RRset is under 100 records, the cumulative count across all RRsets can be 300, 400, or more total records in that one DNS message—pushing it up to the 65 KB limit.

4. Why It’s Becoming Less Effective

- **Minimal ANY Responses**: Modern authoritative servers and DNS providers often do *not* return a full ANY response. They may return a minimal set or refuse ANY queries outright.
- **Rate Limiting**: Many resolvers or authoritative servers throttle or refuse ANY queries due to their amplification potential in DDoS attacks.
- **Cache or Resolver Policies**: Some resolvers do not even cache the entire ANY response; they cache only the records relevant to subsequent queries.

Despite these mitigations, if the attacker runs their own authoritative server (with no such restrictions) and the targeted resolver allows ANY queries, you can still generate *one huge 65 KB response* with a patchwork of multiple RRsets, each under the 100-record threshold.

Key Takeaway

A **qtype=ANY** query can circumvent BIND’s per-RRset 100-record limit by combining *multiple* RRsets in a single response. While each individual RRset may be capped at 100 records, **there is no global cap on the total number of RRsets**. Hence, the attacker can produce an outsized 65 KB DNS message—leading to cache flooding and eviction of legitimate entries—*if* modern protections (like minimal ANY or query blocking) are not in place.

DNSSEC Overhead: If the zone is signed, each RRset might also have RRSIGs, plus possibly DS or DNSKEY records in the Additional/Authority sections. This extra overhead (signatures, keys) can push the total packet size closer to 65 KB.

Focus here. explain how it by-passes the 100 record cap?

Below is a concise explanation of *why DNSSEC overhead (RRSIGs, DNSKEYs, DS, etc.) can push a response toward 65 KB without violating BIND's 100-record-per-RRset limit*.

---

1. BIND's 100-Record Cap Is Per RRset

- **What it is**: BIND typically enforces a maximum number of records per *one* RRset (e.g., you cannot have more than 100 `A` records in `big.example.com A`).
- **What it is *not***: This limit *does not* cap the *total* number of records across *different* RRsets. Each distinct RRset (e.g., `A`, `AAAA`, `TXT`, `RRSIG(A)`, `RRSIG(AAAA)`, `DNSKEY`) is subject to its own 100-record limit, but there is no global limit on the combined total.

---

2. DNSSEC Overhead Creates *Additional* RRsets

When a zone is signed:

1. **RRSIG**:
   - Each ordinary RRset (e.g., your `A` RRset) gets its own RRSIG(s).
   - These signatures form *separate* RRSIG RRsets. Thus, if you have `A` records and `AAAA` records, you'll also have `RRSIG(A)` and `RRSIG(AAAA)` as *two* additional RRsets.
2. **DNSKEY**:
   - Your zone's public keys form a `DNSKEY` RRset, distinct from all other record types.
   - You might have multiple large DNSKEYs.
3. **DS**:
   - If you're returning DS records (in the Authority section), that's yet another RRset.
4. **NSEC/NSEC3**:
   - Typically only one record per "covered name," but each is *still* a separate RRset that may appear in the response.

> **Important**: Because each RRSIG, DNSKEY, DS, NSEC(3) set is *its own* RRset, you do not violate the 100-record cap simply by adding them. Rather, you end up with *multiple*

> RRsets in one response. Each can have up to 100 records on its own.

---

3. Multiple RRsets Summed Up in *One* Response

When the resolver queries (for example) `ANY` or requests DNSSEC for a record, the authoritative server can place *all* these RRsets— `A`, `AAAA`, `TXT`, `DNSKEY`, multiple RRSIG sets, DS, etc.—into the same DNS response:

1. **A** RRset (up to 100 records) +
2. **RRSIG(A)** RRset (some number of RRSIG records) +
3. **AAAA** RRset (up to 100 records) +
4. **RRSIG(AAAA)** … +
5. **DNSKEY** RRset (could be multiple keys) +
6. **RRSIG(DNSKEY)** … +
7. **DS** RRset … +
8. Possibly more types (TXT, MX, NSEC, NSEC3) with their own RRSIG sets…

Even if *each* RRset individually has under 100 records, the *total* number of records across *all* these RRsets can exceed several hundred. This can inflate the overall DNS message to **near 65 KB** (the EDNS0 limit)—all without hitting the per-RRset record cap.

---

4. Large RDATA Further Boosts the Packet Size

In DNSSEC:

- **RRSIG Records** can be large if using big RSA keys (e.g., 4096-bit). Each signature might be hundreds or thousands of bytes.
- **DNSKEY Records** can similarly be hundreds or thousands of bytes for each key.
- Having multiple keys produces multiple RRSIGs *per* RRset, compounding the total size.

So even with *fewer than 100 records per RRset*, large RDATA in DNSSEC records leads to *huge* total byte counts—easily pushing the response to the ~65 KB maximum.

---

5. Bottom Line

How DNSSEC Overhead Bypasses the 100-Record Cap

- The 100-record limit applies **only to an individual RRset** (e.g., `A`, `AAAA`, or `DNSKEY`).
- DNSSEC overhead (RRSIGs, DNSKEYs, DS, NSEC) appears as *additional distinct RRsets* in the same response. Each is allowed up to 100 records, so you end up with *many* RRsets in a single 65 KB reply.
- Because the resolver typically *caches* everything in that single (valid) response, it can evict legitimate records in the process—thus enabling a cache-flushing or cache-pollution attack.